

μ T-Kernel3.0
micro:bit v2 向け
構築手順書

Version.01.00.00

2022.05.17

更新履歴

版数(日付)	内 容
1.00.00 (2022.05.17)	● 初版

目次

1. 概要.....	4
1.1 目的.....	4
1.2 対象 OS およびハードウェア.....	4
1.3 対象開発環境.....	4
2. C コンパイラ.....	5
2.1 GCC バージョン.....	5
2.2 動作検証時のオプション.....	5
2.3 インクルードパス.....	5
2.4 標準ライブラリ.....	5
3. 開発環境と構築手順.....	6
3.1 共通開発ソフトの準備.....	6
3.2 Make を使用したビルド方法.....	6
3.2.1 ビルド環境の準備.....	6
3.2.2 プロジェクトのビルド.....	8
3.3 Eclipse を使用した構築手順.....	8
3.3.1 Eclipse の準備.....	8
3.3.2 プロジェクトの作成.....	8
3.3.3 プロジェクトのビルド.....	10
4. アプリケーションプログラムの作成.....	11
5. 実機でのプログラム実行.....	12
5.1 PyOCD のインストール.....	12
5.2 実機の FlashROM の消去.....	13
5.3 Eclipse によるプログラムの実行.....	13

1. 概要

1.1 目的

本書は、micro:bit v2 向け μ T-Kernel3.0 の開発環境の構築手順を記す。

以降、本ソフトとは前述の μ T-Kernel3.0 のソースコードを示す。

1.2 対象 OS およびハードウェア

本書は以下を対象とする。

分類	名称	備考
OS	μ T-Kernel3.00.05	TRON フォーラム
実機	micro:bit v2	Micro:bit Educational Foundation
搭載マイコン	nRF52833	Nordic Semiconductor

1.3 対象開発環境

本ソフトはC言語コンパイラとして、GCC(GNU Compiler)を前提とする。

ただし、本ソフトはハードウェア依存部を除けば、標準のC言語で記述されており、他のC言語コンパイラへの移植も可能で可能である。

2. C コンパイラ

2.1 GCC バージョン

本ソフトの検証に用いた GCC のバージョンを以下に記す。

GNU Arm Embedded Toolchain
Version 10.3-2021.10

2.2 動作検証時のオプション

本ソフトの動作検証時のコンパイラ及びリンカのオプションを示す。なお、オプションは、開発するアプリケーションに応じて適したものを指定する必要がある。

最適化オプションは、検証時には-O0 を設定している。
リンクタイム最適化-flto (Link-time optimizer) については動作を保証しない。

その他の主なオプションを以下に示す。

コンパイルオプション

-mcpu=cortex-m4 -mthumb -ffreestanding -std=gnu11
-mfpu=fpv4-sp-d16 -mfloat-abi=hard

リンクオプション

-mcpu=cortex-m4 -mthumb -ffreestanding -nostartfiles
-mfpu=fpv4-sp-d16 -mfloat-abi=hard

2.3 インクルードパス

μT-Kernel3.0 のソースディレクトリ中の以下のディレクトリを、ビルド時のインクルードパスに指定する。

ディレクトリパス	内容
¥config	コンフィギュレーションファイル
¥include	共通ヘッダファイル
¥kernel¥knlinc	カーネル内共通ヘッダファイル

¥kernel¥knlinc は OS 内部でのみ使用するヘッダファイルである。ユーザプログラムでは、¥config と ¥include のヘッダファイルのみを使用する。

2.4 標準ライブラリ

本ソフトは基本的にはコンパイラの標準ライブラリを使用しない。ただし、指定するコンパイルオプションなどにより、コンパイラが暗黙に使用する場合がある。コンパイラによる暗黙の使用がなければ、標準ライブラリは不要である。リンカオプションで-nostdlib が指定可能となる。ただし、アプリケーションで使用している場合はこの限りではない。

3. 開発環境と構築手順

本ソフトをビルドするための開発環境の準備と構築手順を説明する。

本ソフトは極力、特定の開発環境に依存しないように作られている。ここでは例として、Windows の PC において、自動ビルドツール Make を使用する場合と、オープンソースの統合開発環境 Eclipse を使用する場合を説明する。

なお、ここに示す開発環境や構築手順はあくまで例であり、ユーザそれぞれの環境などによって差異がある場合がある

3.1 共通開発ソフトの準備

C コンパイラなど共通の開発ツールをインストールする。これらは Eclipse でも Make でも使用する。

(1) C コンパイラのインストール

GCC コンパイラ式を以下からダウンロードする。

GNU Arm Embedded Toolchain

<https://developer.arm.com/tools-and-software/open-source-software/developer-tools/gnu-toolchain/gnu-rm>

本稿作成時に検証したバージョンは以下の通り。

Version 10.3-2021.10

gcc-arm-none-eabi-10.3-2021.10-win32.zip

ダウンロードした zip ファイルを任意の場所に展開する。

(2) 開発ツールのインストール

GCC toolchain を使用するためのツール式 (make など) を以下からダウンロードする。

xPack Windows Build Tools

<https://github.com/xpack-dev-tools/windows-build-tools-xpack/releases>

本稿作成時に検証したバージョンは以下の通り。

Version 4.3.0-1

xpack-windows-build-tools-4.3.0-1-win32-x64.zip

ダウンロードした zip ファイルを任意の場所に展開する。

3.2 Make を使用したビルド方法

3.2.1 ビルド環境の準備

(1) 実行パスの設定

Windows のコマンドシェル (PowerShell またはコマンドプロンプト) から、GCC および Make が実行可能となるように、環境変数 path に GCC を展開したディレクトリ内の %bin ディレクトリのパスおよび、xPack Windows Build Tools を展開したディレクトリ内の %bin ディレクトリのパスを追加設定する。

この設定は、Eclipse を使用する場合も必要である。

[Win]+[Q] を押し「環境変数を編集」と入力すると、環境変数のウィンドウが開くので、そこで設定できる。

コマンドシェルから GCC (arm-none-eabi-gcc) および make コマンドが実行可能であることを確認する。

(2) makefile の設定

本ソフトのソースコード中の Make 用ビルドディレクトリ (build_make) に makefile が格納されている。

ディレクトリ (build_make) の内容を以下に示す。

名称	説明
makefile	μ T-Kernel 3.0 のビルド規則 (ルート)
microbit.mk	micro:bit 用のビルド規則
上記以外の *.mk	他機種用のビルド規則 (※)
/mtkernel_3	Make 作業用ディレクトリ
/setting	各種設定

※ 本書の説明では使用しない。

makefile ファイルの先頭の以下の定義を確認する。

定義名	初期値	説明
EXE_FILE	mtkernel_3	ビルドする実行ファイル名
TARGET	_MICROBIT_	対象とするハードウェア micro:bit の場合は「_MICROBIT_」とする。

また、microbit.mk の先頭の以下の定義を必要に応じて変更する。

定義名	初期値	説明
GCC	arm-none-eabi-gcc	C コンパイラのコマンド名
AS	arm-none-eabi-gcc	アセンブラのコマンド名
LINK	arm-none-eabi-gcc	リンカのコマンド名
CFLAGS	省略 (※)	C コンパイラのオプション
ASFLAGS	省略 (※)	アセンブラのオプション
LFLAGS	省略 (※)	リンカのオプション
LINKFILE	省略 (※)	リンク定義ファイル

※ microbit.mk ファイルの記述を参照

他のファイルについては OS のソースコードの変更が無い限り、変更する必要はない。

ただし、ユーザプログラムの追加等については、それぞれ対応するビルド規則を記述する必要がある。

また `app_sample` ディレクトリ下のアプリケーションについては以下のファイルでビルド規則が記述されている。

```
build_make¥mtkernel_3¥app_sample¥subdir.mk
```

`app_sample` ディレクトリにソースファイルを追加しても対応可能なビルド規則となっているが、サブディレクトリには対応していない。サブディレクトリを作成する場合はビルド規則の記述を変更する必要がある。

3.2.2 プロジェクトのビルド

Windows のシェル (PowerShell またはコマンドプロンプト) 上で、`build_make` ディレクトリをカレントディレクトリとし、以下のコマンドを実行する。

```
make all
```

ビルドが成功すると、`build_make` ディレクトリ下に、実行コードの ELF ファイルが生成される。ELF ファイルの名称は `EXE_FILE` で指定した名称である (初期値では `mtkernel_3.elf` が生成される)。

また、以下のコマンドを実行すると、ELF ファイルおよびその他の中間生成ファイルが削除される。

```
make clean
```

3.3 Eclipse を使用した構築手順

3.3.1 Eclipse の準備

(1) Eclipse のインストール

Eclipse IDE for Embedded C/C++ Developers を以下からダウンロードする。

<https://www.eclipse.org/downloads/packages/release/2022-03/r/eclipse-ide-embedded-cc-developers>

本稿作成時に検証したバージョンは以下の通り。

`eclipse-embedcpp-2022-03-R-win32-x86_64.zip`

ダウンロードした zip ファイルを任意の場所に展開する。

(2) Eclipse の起動

Eclipse の実行は、zip ファイルを展開したディレクトリ中の以下のプログラムを起動する。

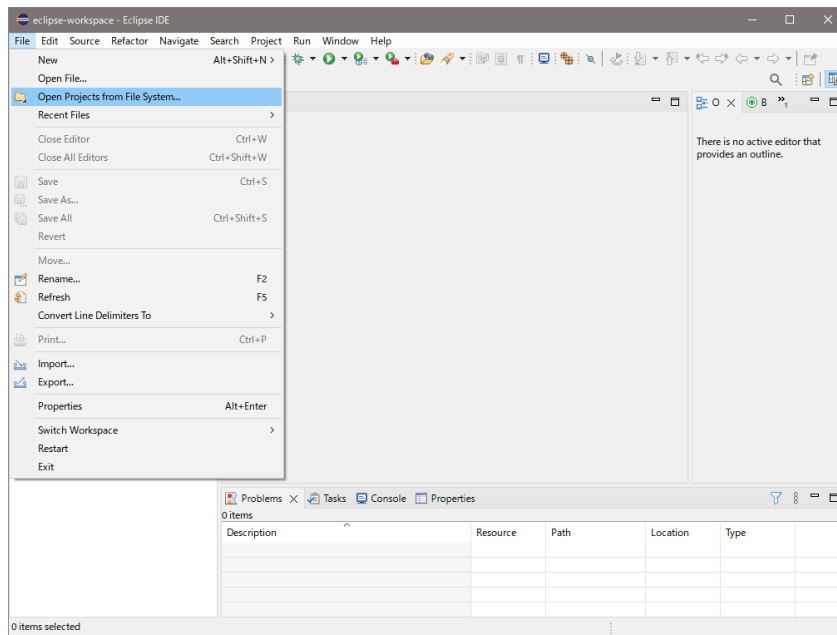
<展開したディレクトリ>¥eclipse¥eclipse.exe

Eclipse の初回起動時は、指示に従いワークスペースを作成する。ワークスペースは、Eclipse の各種設定などが保存される可能的な作業場である。

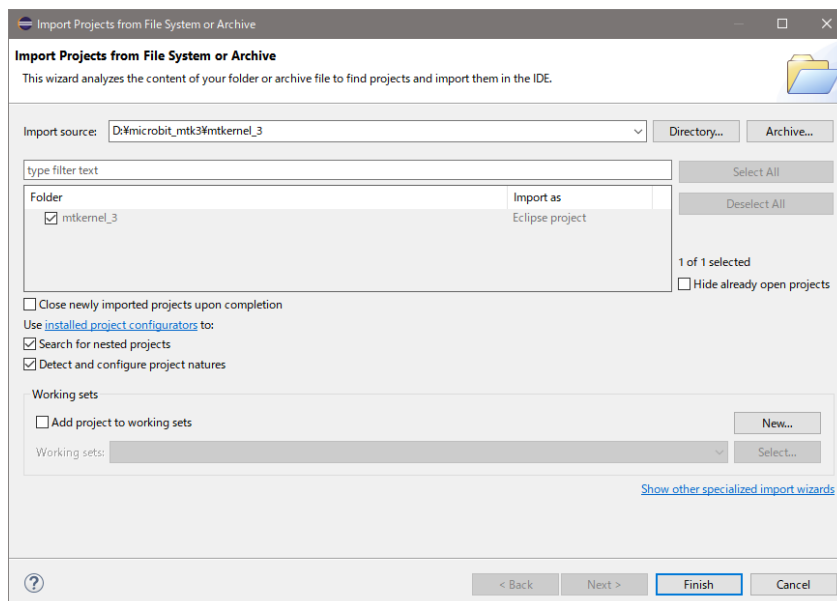
3.3.2 プロジェクトの作成

Eclipse にて以下の手順で本ソフトのプロジェクトを作成する。

- (1) メニューの File - Open Projects from File System... を実行する。



- (2) Import source の Directory... を押し、本ソフトのソースコードのルートとなるディレクトリを指定し、Finish を押す。



指定するディレクトリは、build_make の親となるディレクトリである。
上記例では、build_make の位置が D:\microbit_mtk3\mtkernel_3\build_make であるので、指定するのは D:\microbit_mtk3\mtkernel_3 となる。

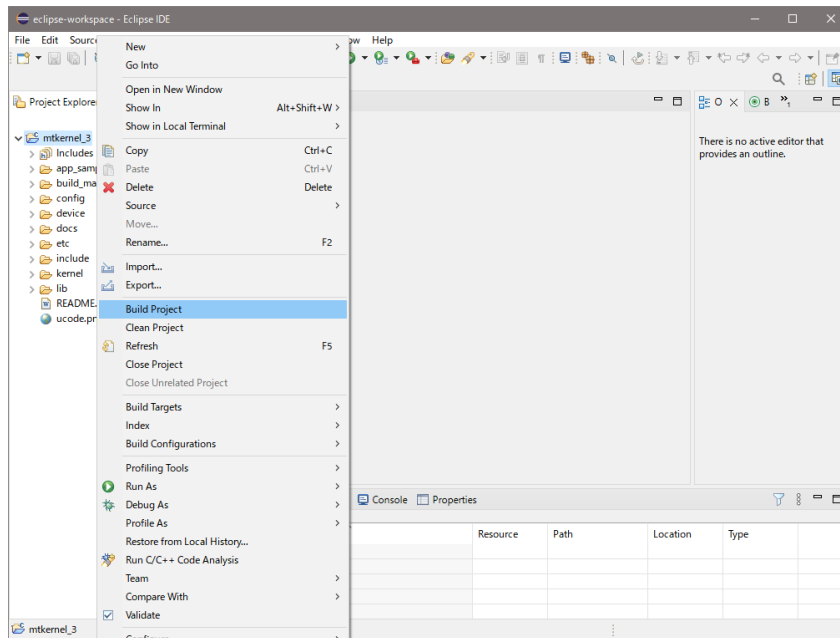
- (3) Project Explorer に mtkernel_3 が表示されているのを確認する。

このプロジェクトは、makefileにしたがってビルドする。
そのため、「3.2.1 ビルド環境の準備」にしたがって、ビルド環境を整える必要がある。特に、環境変数 path の設定を必ず行うこと。

3.3.3 プロジェクトのビルド

Project Explorer の mtkernel_3 上で右クリックし、メニューの Build Project を実行する。

コンパイルが実行されるので、正常に終わるのを確認する。



4. アプリケーションプログラムの作成

アプリケーションプログラムは、OS とは別にアプリ用のディレクトリを作成して、そこにソースコードを置き、OS と一括でコンパイル、リンクを行う。

公開されている μ T-Kernel3.0 のソースコードには、/app_sample ディレクトリにサンプルのアプリケーションのソースコードが含まれている。
ソースコードは以下のファイルに記述されている。

```
/app_sample/app_main.c
```

サンプルのアプリケーションは、初期タスクから二つのタスクを生成、実行し、T-Monitor 互換ライブラリを使用してシリアル出力にメッセージを出力する簡単なプログラムである。これをユーザの作成したアプリケーションプログラムに置き換えればよい。

アプリケーションプログラムには、usermain 関数を定義する。OS は起動後に初期タスクから usermain 関数を実行する。詳細は μ T-Kernel3.0 共通実装仕様書「5.2.3 ユーザ定義メイン関数 usermain」を参照のこと。

アプリケーションから OS の機能を使用する場合は、以下のようにヘッダファイルのインクルードを行う。

```
#include <tk/tkernel.h>
```

T-Monitor 互換ライブラリを使用する場合は、さらに以下のインクルードが必要である。

```
#include <tm/tmonitor.h>
```

μ T-Kernel3.0 の機能については、 μ T-Kernel3.0 仕様書を参照のこと。

5. 実機でのプログラム実行

プログラムを実機上で実行する方法を、Eclipse を使用した例で説明する。
Eclipse の開発環境から、実機に実行コードを転送しデバッグを行う。
開発用 PC と実機は、USB ケーブルで接続する。

5.1 PyOCD のインストール

(1) Python を次の Web サイトからダウンロードする。

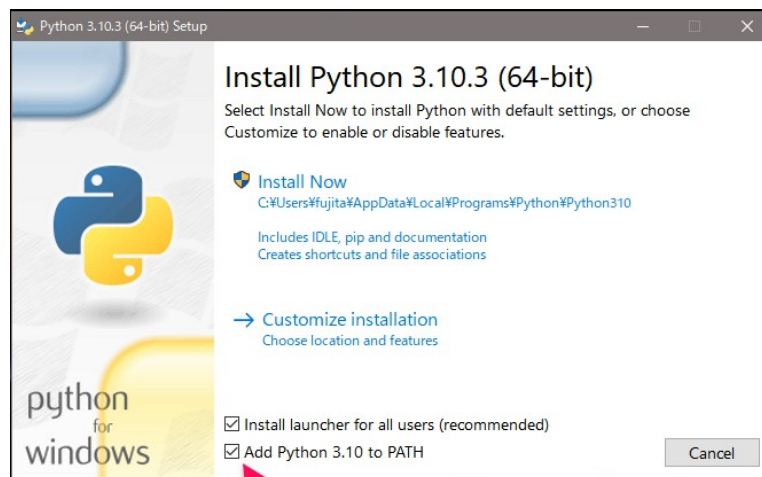
<https://www.python.org/>

本稿作成時に検証したバージョンは以下の通り。

Version 3.10.3

python-3.10.3-amd64.exe

(2) ダウンロードしたインストーラを実行し、Python をインストールする。
下図のように PATH の追加を指定し、「Install Now」でインストールする。



(3) コマンドプロンプトを開き、次のコマンドで PyOCD をインストールする。

```
python -mpip install -U pyocd
```

PyOCD が実行できることを確認する。

```

C:\>pyocd
usage: pyocd [-h] [-V] [--help-options] ...
PyOCD debug tools for Arm Cortex devices
options:
  -h, --help            show this help message and exit
  -V, --version          show program's version number and exit
  --help-options        Display available session options.
subcommands:
  commander (cmd)
                        Interactive command console.
  erase                 Erase entire device flash or specified sectors.
  load (flash)         Load one or more images into target device memory.
  gdbserver (gdb)      Run the gdb remote server(s).
  json                 Output information as JSON.
  list                 List information about probes, targets, or boards.
  pack                 Manage CMSIS-Packs for target support.
  reset                Reset a target device.
  server               Run debug probe server.
  rtt                  SEGGER RTT Viewer.
C:\>

```

5.2 実機のFlashROMの消去

micro:bitを初めて使用する際は、購入時にFlashROMに書き込まれているプログラムを消去する必要がある。

PCとmicro:bitをUSBケーブルで接続する。

コマンドプロンプトを開き、下記のコマンドを実行してFlashROMを消去する。

```
pyocd erase --mass
```

実行例

```

C:\>pyocd erase --mass
0002357 I Mass erasing device... [eraser]
0002570 I Successfully erased. [eraser]
C:\>

```

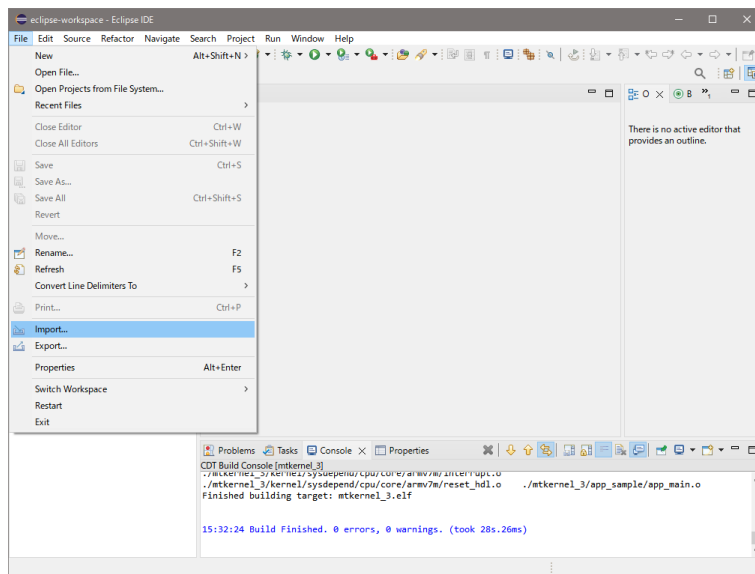
【参考】FlashROMを購入時の状態に戻す場合は、以下を参照。

<https://support.microbit.org/support/solutions/articles/19000021613-first-experience-demo-program>

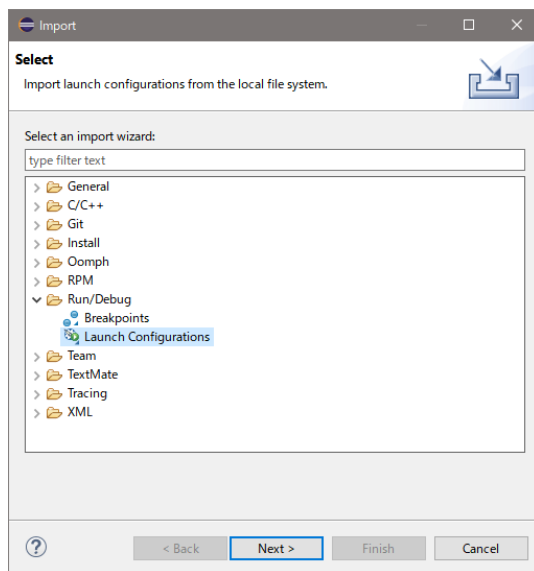
5.3 Eclipseによるプログラムの実行

(1) Eclipseにデバッガの設定を読み込む。

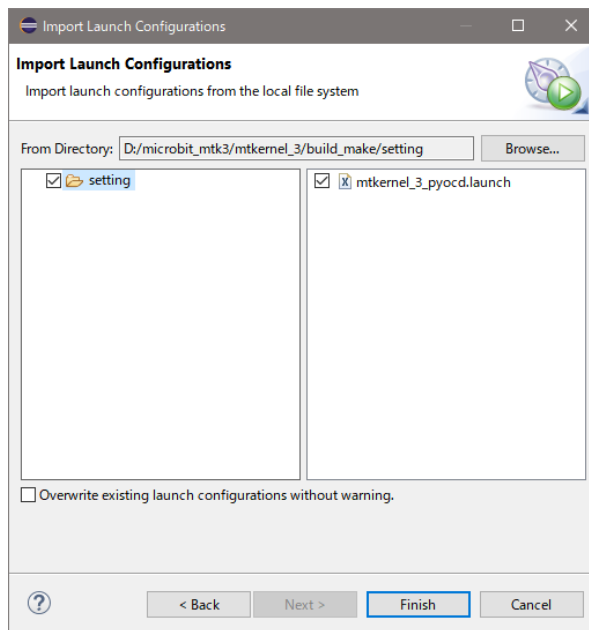
メニューの File - Import... を実行する。



Run/Debug – Launch Configurations を選択し、Next を押す。

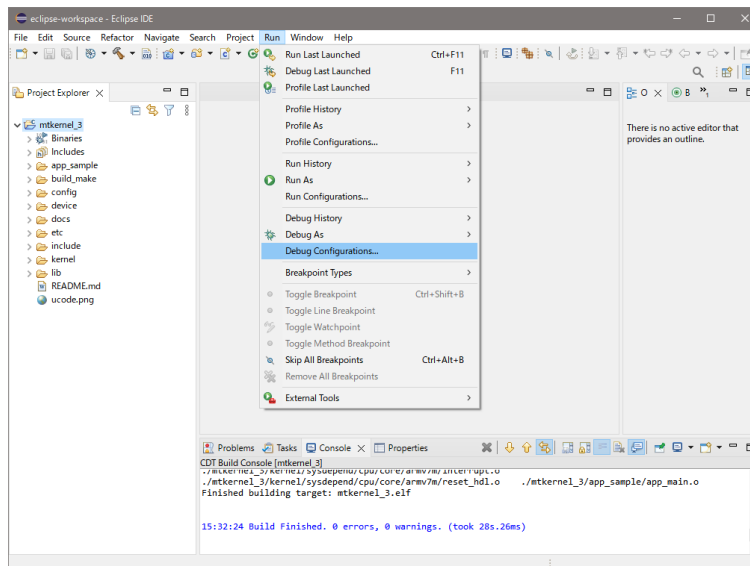


From Directory に本ソフトのソース内の build_make/setting を指定し、setting および mtkernel_3_pyocd.launch にチェックを入れ、Finish を押す。

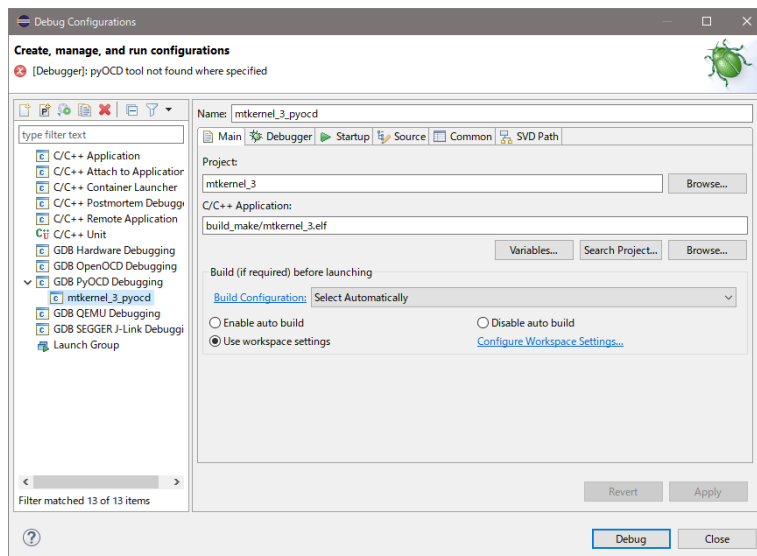


- (2) ターミナルソフトを起動する。
 micro:bit を PC に USB ケーブル で接続する。
 ターミナルソフト (任意) を起動し、micro:bit の UART に接続する。
 COM 番号は、デバイスマネージャーで確認する。
 通信設定 115200bps / 8bit / 1stop-bit / no-parity / no-flowcontrol

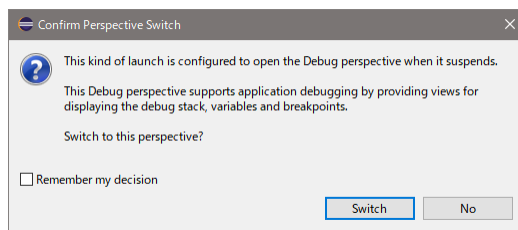
- (3) デバッガを起動する。
 メニューの Run - Debug Configurations... を実行する。



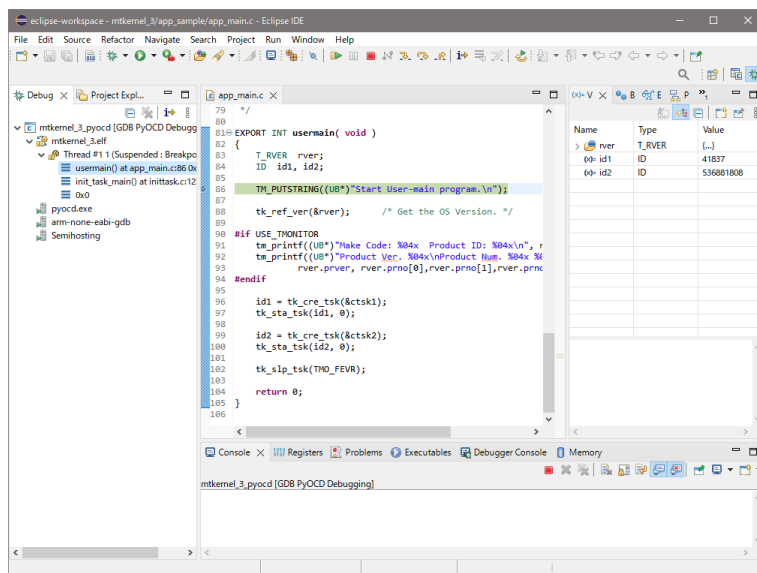
GDB PyOCD Debugging の mtkernel_3_pyocd を選択し、Debug を押す。



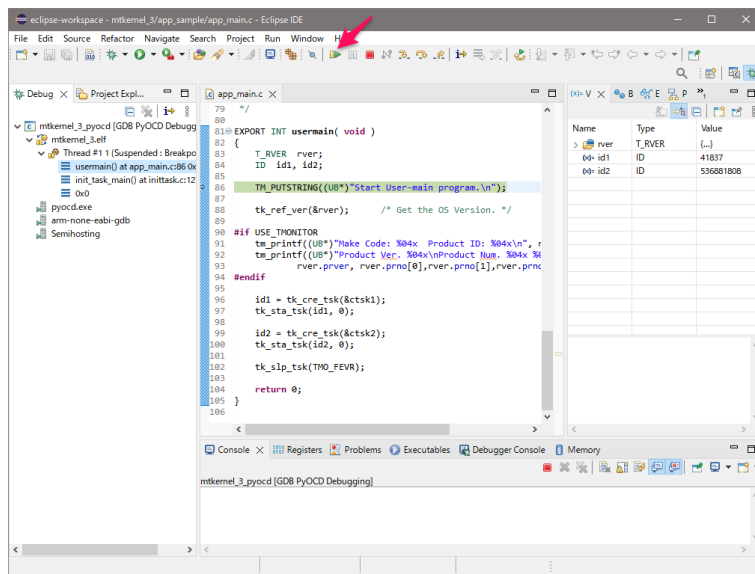
次のパネルが表示されたら、Switch を押す。



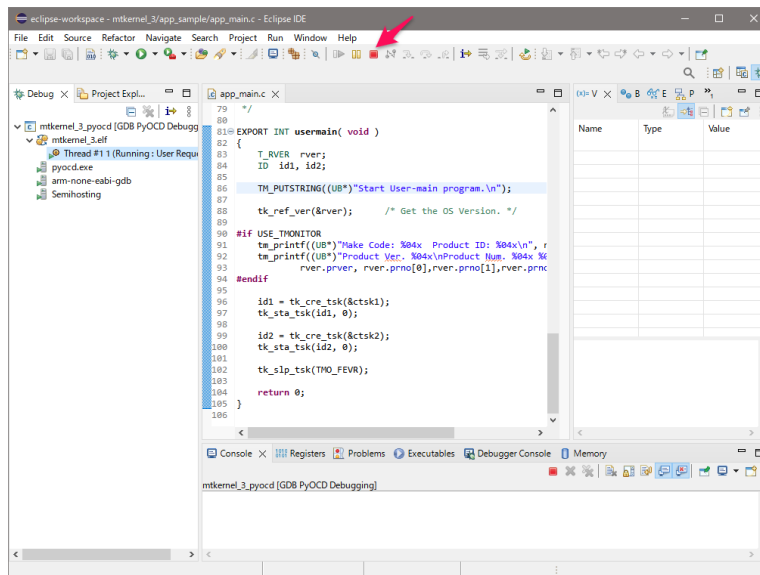
デバッガが起動する。
プログラムがFlashROMに書き込まれ、実行を開始する。
プログラムは、`usermain()` の先頭で止まる。



- (4) デバッグを行う。
Run – Resume でプログラムが実行される。



Run – Terminate でプログラムを停止する。



以上